Electron Dual-Monitor POS Setup

Automatic secondary monitor detection and fullscreen customer display using Electron.

Features

✅ **Automatic Secondary Monitor Detection** - Customer display opens automatically on second screen ✅ **Kiosk Mode** - Customer display runs fullscreen with no browser chrome ✅ **Persistent Display** - Customer window can't be accidentally closed ✅ **Hot-Reload** - Connects to Next.js dev server for development ✅ **Production Ready** - Can be packaged as standalone application

Prerequisites

1. **A running app server to connect to** — this can be a local dev server (`npm run dev`, accessible at `http://localhost:8080`) or any remote server on the network. Unlike earlier versions of this app, Electron no longer assumes `localhost` — see **Multi-Server Support** below for how a server is selected.
2. **Second monitor connected** (for customer display)

Installation

```
cd electron
npm install
```

Usage

Development Mode

Option 1: Auto-detect POS type

```
npm start
```

Opens the default POS (restaurant) on primary monitor and customer display on secondary.

Option 2: Specify POS type

```
# Restaurant POS
npm run start:restaurant

# Grocery POS
npm run start:grocery

# Hardware POS
```

```
npm run start:hardware

# Clothing POS
npm run start:clothing
```

Option 3: With DevTools

```
npm run dev
```

Opens with Chrome DevTools enabled for debugging.

Environment Variables

Create a `.env` file in the `electron/` directory:

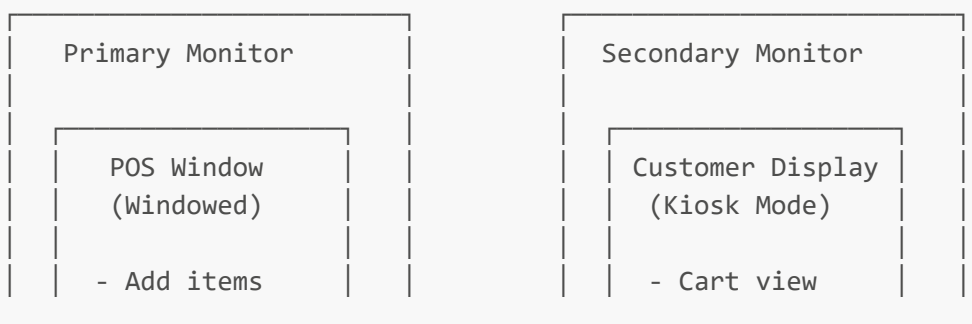
```
# POS type (restaurant, grocery, hardware, clothing)
POS_TYPE=restaurant

# Business ID (optional, defaults to 'default-business')
BUSINESS_ID=biz_123abc

# Terminal ID (optional, auto-generated if not provided)
TERMINAL_ID=terminal-001

# Development mode (opens DevTools)
NODE_ENV=development
```

Configuration

Edit `main.js` to customize:

- **Window sizes** (lines 28-31, 47-50)
- **POS URL** (line 40)
- **Customer display URL** (line 66)
- **Kiosk mode behavior** (line 48)

How It Works



```
│  │  - Checkout      │◄──┼──────────┼──►│  - Ads/marketing │   │
│  │  - Settings      │   │   Sync   │   │  - Fullscreen    │   │
│  └──────────────────┘   │          │   └──────────────────┘   │
└─────────────────────────┘          └──────────────────────────┘
```

1. **App starts** → Detects all monitors
2. **Primary window** → Opens POS on main display (windowed)
3. **Secondary window** → Opens customer display on second monitor (fullscreen kiosk)
4. **Sync** → BroadcastChannel for instant communication (same device)

Multi-Server Support

This Electron shell can be registered against several different app servers — different companies, or a test server alongside a real production one — and switch between them, each with a **completely separate login**. Server B never sees Server A's cookies or session, even mid-shift on the same kiosk.

First launch (or after removing the last-used server): a local "Select Server" screen appears — no remote content is loaded until one is chosen.

Adding a server:

1. Click **+ Add Server** (the first time, you'll be asked to set a local PIN — this protects add/remove only, not switching between already-registered servers).
2. Enter a label, the server's IP address, and an **admin** email/password for that server.
3. Click **Test Connection** — this performs a real connection attempt (not just a ping): validates the IP, checks reachability, and actually signs in with the given credentials to confirm they're a real admin account on that specific server. If the server presents a self-signed certificate that isn't already trusted, you'll be shown its details and asked to explicitly confirm trust before the test proceeds — that certificate is then pinned to this one server going forward, not blanket-trusted by hostname.
4. The credentials are used only for this one-time check and are never stored — whoever actually uses the kiosk logs in fresh, with their own account, once the server is open.
5. **Save Server** is only enabled once a test has passed for the exact values currently in the form — editing anything afterward requires re-testing.

By default, a bare IP expands to `https://<ip>:8080` (this app's standard port). Use **Advanced: use a full URL instead** for a custom port or a server behind a different setup.

Switching servers: click **Connect** next to any registered server, or use the **Server** → **Switch Server…** menu item (also bound to `Ctrl+Shift+S` / `Cmd+Shift+S`, which works even with the menu bar hidden in kiosk mode). Switching briefly tears down and recreates both windows — this is deliberate, not a bug, since a session partition can't be swapped on a live window without risking leaking state between servers.

Next launch automatically reopens whichever server was last used — the picker only appears again if that server can't be reached (with the failure and its support contact shown right there), if it's been removed, or if you explicitly switch back to it.

Deploying to a Remote Workstation

This app no longer has to run on the same machine as the server — see **Multi-Server Support** above. For an actual kiosk (not a dev machine), build a standalone installer here and copy just that installer to the workstation — it needs no Node.js, no npm, no checkout of this repo at all.

Before building for real distribution, two things this repo doesn't ship with:

- **Icon files** — `icon.ico` / `icon.icns` / `icon.png` in this folder, referenced by the `build` config in `package.json`. Without them, `electron-builder` falls back to its own placeholder — fine for testing, not for handing to a business.
- **Code signing** — without it, the installer is unsigned and Windows SmartScreen will show an "unrecognized app" warning on first run (click through "More info → Run anyway"). Expected for an internal tool without a signing certificate, not a bug.

The Windows build (`package.json`'s `nsis` config) is set up as a **single, per-machine install** — one install, available to every Windows user account on that workstation, not just whoever ran the installer. It's a one-click install (no wizard, nothing to configure) that will show a **UAC elevation prompt** during install — expected, since a per-machine install writes to Program Files and the all-users shortcut locations. If you'd rather have a per-user install (no elevation prompt, but only usable by the account that installed it) or the traditional wizard UI, both are separate `nsis` options — ask before changing this, it affects how every future install of this app behaves.

Production Build

Windows

```
npm run build:win
```

Creates installer in `dist/` folder.

macOS

```
npm run build:mac
```

Creates DMG in `dist/` folder.

Linux

```
npm run build:linux
```

Creates AppImage in `dist/` folder.

Troubleshooting

"No secondary display detected"

- Connect second monitor before starting Electron
- Check display settings in your OS
- Restart Electron after connecting monitor

"Couldn't reach " / stuck on the server picker

- Confirm the server is actually running and reachable from this machine (try its address in a regular browser first)
- If it just moved or changed IP, use **+ Add Server** to register the new address rather than editing the old entry — each registered server is tied to a specific URL
- Check the support-contact number shown on the failure banner, if one was set when this server was registered

Customer display not fullscreen

- This is normal - kiosk mode handles it automatically
- Window will fill entire secondary screen
- Use `Esc` key to exit kiosk mode (for testing only)

DevTools not opening

- Set `NODE_ENV=development` in `.env` file
- Or use `npm run dev` command

Alternative: Web-Based Auto-Detection

If you don't want to use Electron, the web-based solution uses the **Window Management API**:

1. Modern browsers (Chrome 100+, Edge 100+)
2. Requires user permission prompt
3. Automatically positions on secondary monitor
4. Click the "🖥️ Display" button in POS
5. Allow permission when prompted
6. Window opens on secondary monitor automatically

Next Steps

- **Customize styling** in customer display page
- **Add advertisement management** (Phase 5)
- **Configure business/terminal IDs** via .env
- **Package for deployment** with electron-builder